Dicionários

Dicionários

- **Dicionários** (em Python, **`dict`**) são estruturas de dados que implementam *mapeamentos*.
- Um *mapeamento* é uma coleção de associações entre pares de dados.
  - Um item do par é a chave (*key*) e o outro item é o valor (*value*).
- Um *mapeamento* é uma generalização da ideia de acessar dados por índices, exceto que num mapeamento os índices (ou chaves) podem ser de qualquer tipo *imutável*.
  - Tipo *imutável*: constantes numéricas, strings ou tuplas.

Dicionários: Chaves vs. Índices

- Como representar uma lista telefônica?
  - Uma solução são duas listas, uma de nomes e outra de telefones.
    - Telefone de `nome[i]` armazenado em `telefone[i]`.
    - Acrescentar “Joao” com telefone “(89)99470-7890”:

```
nome.append("Joao")
telefone.append("(86)996470-7890")
```

    - Para encontrar o telefone de “Joao”:

```
fone = telefone[nome.index("Joao")]
```

  - Dicionários tornam isso mais fácil e eficiente.

```
telefone["Joao"] = "(86)996470-7890"
tel = telefone["Joao"]
```

153

Criando dicionários

- Uma constante do tipo dicionário é escrita como
  { chave1 : valor1, chave2 : valor2, ... chaveN : valorN }
- Uma variável do tipo dicionário é definida usando a funçao **`dict()`**.
- Uma variável do tipo dicionário pode ser “indexada” da maneira habitual, isto é, usando colchetes **`[ ]`**.
- O valor associado a uma chave pode ser alterado atribuindo-se àquela “posição” do dicionário.
- Novos valores podem ser acrescentados a um dicionário fazendo atribuição a uma chave ainda não definida.
- Não há ordem definida entre os pares chave/valor de um dicionário.

154

Exemplo

```
>>> notas = {"ana":10.0, "ivo":9.0}
>>> notas["ana"]
    10.0

>>> notas["ivo"]
    9.0

>>> notas["edu"] = 8.00         # inserindo novo item
>>> print(notas)
    {'ana': 10.0, 'ivo': 9.0, 'edu': 8.0}

>>> notas = {"ely":7.00, "alan":6.0}
>>> print(notas)
    {'ely': 7.0, 'alan': 6.0}
```

155

Dicionário não tem ordem?

- As chaves dos dicionários não são armazenadas em ordem.
  - Diferentemente de listas, atribuir a um item de um dicionário não requer que a posição exista previamente.

```
>>> X = []
>>> X[10] = 5                          # Em lista dá ERRO!
    Traceback (most recent call last):
    File "<pyshell#10>", line 1, in <module>
        x[10] = 5
    IndexError: list assignment index out of range

>>> Y = {}
>>> Y[10] = 5                          # OK! Cria-se a chave 10
>>> Y
    {10: 5}
```

156

Função *dict*

- A função **`dict`**(***parâmetros***) define dicionários.
- Onde, ***parâmetros*** pode ser:
  - vazio ou uma **`list`** de tuplas com dois valores (***chave, valor***).

```
d = dict()
d = dict([(1,2),('ana',10.0),('nome','ivo')])
```

ou

  - uma sequência de itens no formato ***chave=valor***.
    - Nesse caso, as chaves têm que ser strings, mas são escritas sem aspas.

```
d = dict(ana=10.0, ivo=9.0, edu=8.0)
```

157

Exemplo

```
>>> d = dict([(1,2),('ana',10.0),('nome','ivo')])
>>> d[1]
    2

>>> d['ana']
    10.0

>>> d = dict(ana=10.0, ivo=9.0, edu=8.0)
>>> d['ivo']
    9.0

>>> d = dict(1=2,3=4)            # ERRO!
SyntaxError: expression cannot contain assignment, perhaps you
meant "==“ ?
```

158

Dicionários: método *update*

- **update**(**dic**) atualiza um dicionário com os elementos de outro.
  - Os itens em **dic** são adicionados um a um ao dicionário original.
  - Pode-se usar a mesma sintaxe da função **dict** pra especificar **dic**.

```
>>> dic = {"ana":10.0, "ivo":9.0}
>>> y = {"z":9, "b":7}
>>> dic.update(y)
>>> print(dic)
    {'ana':10.0, 'ivo':9.0, 'z':9, 'b':7}
```

  - Como a chave "b" já existe, veja o que ocorre a seguir:

```
>>> dic.update(b=8, c="xx")
>>> print(dic)
    {'ana':10.0, 'ivo':9.0, 'z':9, 'b':8, 'c':'xx'}
```


Dicionários: método *pop*

- **pop**(**chave**), para a **chave** informada como parâmetro (obrigatória), remove o item (chave/valor) e retorna o **valor** relativo a **chave**.

```
>>> dic = {"ana":10.0, "ivo":9.0, "edu":8.0}
>>> dic.pop('ivo')
    9.0
>>> dic.pop('ana')
    10.0
>>> print(dic)
    {'edu':8.0}
>>> dic.pop()        # ERRO!
File "<pyshell#31>", line 1, in <module> y.pop()
TypeError: pop expected at least 1 argument, got 0
```

Dicionários: método *popitem*

- **popitem**() remove e retorna o último item (*chave/valor*) inserido no dicionário (último = topo).
  - Pode ser usado para iterar sobre todos os elementos do **dict**.

```
>>> dic = {"ana":10.0, "ivo":9.0, "edu":8.0}
>>> dic.popitem()
    ('edu':8.0)
>>> dic.popitem()
    ('ivo':9.0)
>>> print(dic)
    {'ana': 10.0}
```

161

Dicionários: método *clear*

- **clear**() remove todos os elementos do dicionário. Exemplo:

```
>>>  x = {"ana":10.0, "ivo":9.0}
>>>  y = x
>>>  print(x, y)
     {'ana':10.0, 'ivo':9.0} {'ana':10.0, 'ivo':9.0}
>>>  x.clear()
>>>  print(x, y)
     {} {}          # Os "dois" dicionários x e y são limpos
```

  - Diferente de simplesmente atribuir { } à variável:

```
>>>  x = {"ana":10.0, "ivo":9.0}
>>>  y = x
>>>  x = {}
>>>  print(x, y)
     {} {'ana':10.0, 'ivo':9.0}
```

162

Dicionários: método *copy*

- **copy**() retorna uma cópia "rasa" do dicionário com os mesmos pares chave/valor.
  - Os valores não são cópias, mas referências para os mesmos valores.

```
>>> x = {"ana":[10.0, 8.0], "ivo":[9.0, 7.0]}
>>> y = x.copy()
>>> print(x)     # {"ana":[10.0, 8.0], "ivo":[9.0, 7.0]}
>>> print(y)     # {"ana":[10.0, 8.0], "ivo":[9.0, 7.0]}
>>> y["edu"] = [8.0, 6.0]
>>> x["ana"] += [9.0]
>>> print(x)
    {'ana': [10.0, 8.0, 9.0], 'ivo': [9.0, 7.0]}
>>> print(y)
    {'ana': [10.0, 8.0, 9.0], 'ivo': [9.0, 7.0], 'edu': [8.0, 6.0]}
```

163

Problema no método *copy*

- Perceba abaixo que a inserção da **nota 9.0** em `x["ana"]`, insere também em `y["ana"]`. Para não alterar a lista interna usa-se a função **deepcopy()**.

```
>>> x = {"ana":[10.0, 8.0], "ivo":[9.0, 7.0]}
>>> y = x.copy()
>>> print(x)
    {"ana":[10.0, 8.0], "ivo":[9.0, 7.0]}
>>> print(y)
    {"ana":[10.0, 8.0], "ivo":[9.0, 7.0]}
>>> y["edu"] = [8.0, 6.0]
>>> x["ana"] += [9.0]
>>> print(x)
    {'ana': [10.0, 8.0, 9.0], 'ivo': [9.0, 7.0]}
>>> print(y)
    {'ana': [10.0, 8.0, 9.0], 'ivo': [9.0, 7.0], 'edu': [8.0, 6.0]}
```

164

Função *deepcopy*

- **deepcopy**(), definida no pacote **copy**, retorna uma cópia "profunda" do dicionário com os mesmos pares *chave/valor*.
  - A inserção da **nota 9.0** em `x["ana"]` não se reflete em `y["ana"]`.

```
>>> import copy
>>> x = {"ana":[10.0, 8.0], "ivo":[9.0, 7.0]}
>>> y = copy.deepcopy(x)
>>> print(x)            # {"ana":[10.0, 8.0], "ivo":[9.0, 7.0]}
>>> print(y)            # {"ana":[10.0, 8.0], "ivo":[9.0, 7.0]}
>>> y["edu"] = [8.0, 6.0]
>>> x["ana"] += [9.0]
>>> print(x)            # {'ana': [10.0, 8.0, 9.0], 'ivo': [9.0, 7.0]}
>>> print(y)
    {'ana': [10.0, 8.0], 'ivo': [9.0, 7.0], 'edu': [8.0, 6.0]}
```

165

Dicionários: métodos *items*, *keys* e *values*

- **keys**() retorna uma lista com todas as chaves do dicionário.
- **values**() retorna uma lista com todos os valores do dicionário.
- **items**() retorna uma lista com todos os pares do dicionário em tuplas.
  - Observações: Usa-se as funções **list**() e **tuple**() para converter os resultados em lista e tupla, respectivamente.

```
>>> dic = {"ana":10.0, "ivo":9.0, "edu":8.0}
>>> print(dic.keys())
    dict_keys(['ana', 'ivo', 'edu'])
    tupla = tuple(dic.keys())
>>> print(dic.values())
    dict_values([10.0, 9.0, 8.0])
    lista = list(dic.values())
>>> print(dic.items())
    dict_items([('ana', 10.0), ('ivo', 9.0), ('edu', 8.0)])
```

166

Dicionários: método *fromkeys*

- **fromkeys**(**lista, valor**) retorna um novo dicionário cujas chaves são os elementos de **lista** e cujos valores são todos iguais a **valor**.
  - Se valor não for especificado, o default é **None**. Exemplo:

```
>>> dic = {}.fromkeys([1,2,3])
>>> print(dic)
    {1: None, 2: None, 3: None}
```

- Pode-se usar o nome da classe (**dict**) ao invés de um objeto:

```
>>> dic = dict.fromkeys(["ana","ivo"], 0)
>>> print(dic)
    {'ana': 0, 'ivo': 0}
>>> dic = dict.fromkeys(["ana","ivo"], [10.0, 9.0])
>>> print(dic)
    {'ana': [10.0, 9.0], 'ivo': [10.0, 9.0]}
```

167

Dicionários: método *get*

- **get**(**chave**, **valor**) obtém o conteúdo de **chave**.
  - Não causa erro, se **chave** não existir retorna o **valor** especificado.
  - Se **valor** não for especificado, chave inexistente retorna **None**.

```
>>> dic = {"ana":10.0, "ivo":9.0, "edu":8.0}
>>> x = dic.get("eva")           # x recebe None
>>> print(dic.get("eva"))
    None
>>> x = dic.get("ana")           # x recebe 10.0
>>> print(dic.get("ana"))
    10.0
>>> x = dic.get("eva","N/A")     # x recebe ,"N/A"
>>> print(dic.get("eva","N/A"))
    N/A
```

168

Formatando com dicionários

- O operador de formatação quando aplicado a dicionários requer que os valores das chaves apareçam entre parênteses antes do código de formatação.
- O conteúdo armazenado no dicionário sob aquela chave é substituído na string de formatação. Exemplo:

```
>>> dic = {"ana":10.0, "ivo":9.0}
>>> s = "%(ana).2f e %(ivo).2f"
>>> print(s % dic)
    10.00 e 8.00
```

169